

Practical-6

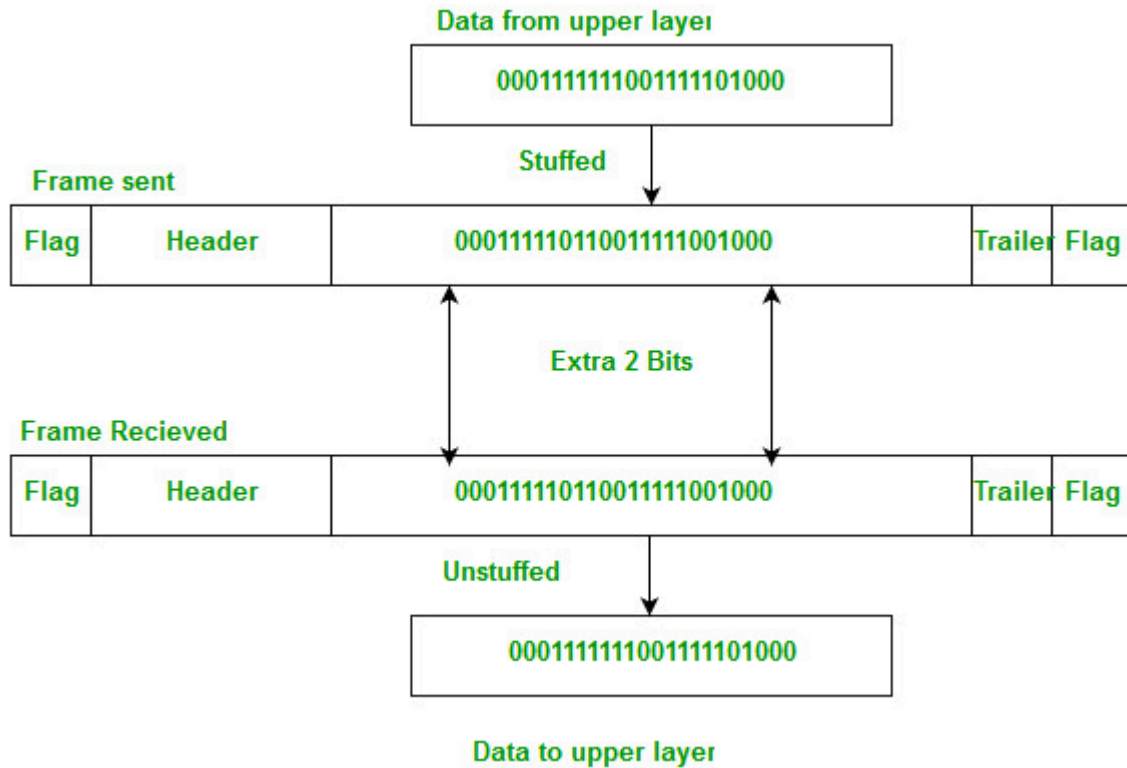
AIM:Write a C program which demonstrates the concept of bit stuffing.

Bit stuffing is a technique used in computer networks to ensure data is transmitted correctly. The data link layer is responsible for framing, which is the division of a stream of bits from the network layer into manageable units (called frames). Frames could be of fixed size or variable size. In variable-size framing, we need a way to define the end of the frame and the beginning of the next frame. A **Bit stuffing** is the insertion of non-information bits into data. Note that stuffed bits should not be confused with overhead bits. **Overhead bits** are non-data bits that are necessary for transmission (usually as part of headers, checksums, etc.).

How Bit Stuffing Work?

Below are the working of bit stuffing in computer network:

- **Sender Side:** While sending data if the sender detects a sequence of bits that matches a special control pattern like five consecutive 1s it inserts an extra bit usually a 0 into the data stream to break the sequence.
- **Receiver Side:** The receiver gets the data and removes the extra stuffed bit whenever it detects a specific bit pattern. This restores the data to its original form.



Applications of Bit Stuffing

1. Synchronize several channels before [multiplexing](#).
2. Rate-match two single channels to each other.
3. Run length limited coding.

Run length limited coding : To limit the number of consecutive bits of the same value(i.e., binary value) in the data to be transmitted. A bit of the opposite value is inserted after the maximum allowed number of consecutive bits.

Bit stuffing technique does not ensure that the sent data is intact at the receiver side (i.e., not corrupted by transmission errors). It is merely a way to ensure that the transmission starts and ends at the correct places.

Code:

```
#include <stdio.h>

int main() {
    char data[100], stuffedData[200];
    int i, j = 0, count = 0;

    printf("Enter the bit stream: ");
    scanf("%s", data);

    for (i = 0; data[i] != '\0'; i++) {
        stuffedData[j++] = data[i];

        if (data[i] == '1') {
            count++;
            if (count == 5) {
                stuffedData[j++] = '0'; // Stuff a 0 after five consecutive 1's
                count = 0;
            }
        } else {
            count = 0;
        }
    }

    stuffedData[j] = '\0';

    printf("Original Bit Stream : %s\n", data);
    printf("Bit Stuffed Stream : %s\n", stuffedData);

    return 0;
}
```

Output:

```
Enter the bit stream: 1111101111110  
Original Bit Stream : 1111101111110  
Bit Stuffed Stream  : 111110011111010
```

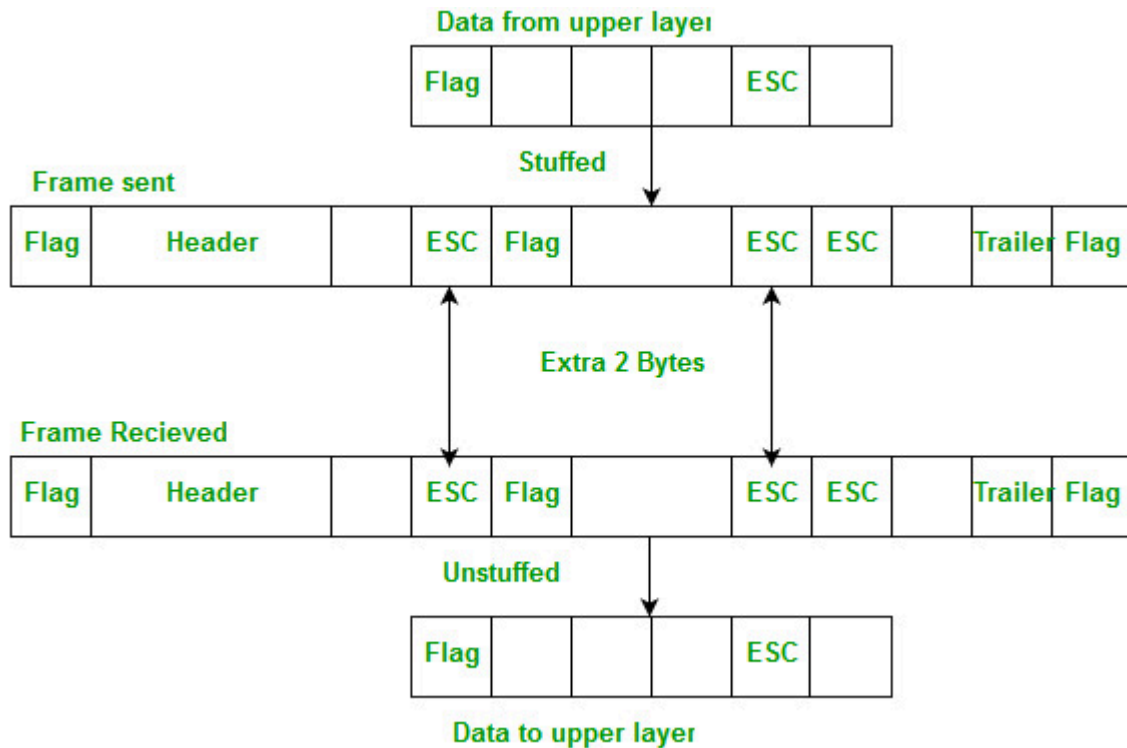
Practical-7

AIM: Write a C program which demonstrates the concept of byte stuffing.

Byte stuffing is a technique used in data communication to assure that unique control characters under a data stream are not wrongly interpreted as part of the actual data. This method is commonly used in communication protocols that use particular byte sequences to signal the start and end of data frames.

How Byte Stuffing Works?

In several communication protocols, particular byte sequences are allocated for control goals, such as analyzing the start or end of a frame. For example, a protocol might use the byte 0x7E to indicate the start and end of a frame. Although, if this byte appears naturally in the data being transmitted, it could cause the receiver to accidentally interpret it as the end of the frame, causing errors.



Applications

Byte stuffing is mostly used in protocols like [PPP](#) (Point-to-Point Protocol) and in sequential communication protocols. By assuring that control characters inside the data are suitably managed, byte stuffing helps keep clear communication, avoiding data loss or error.

Bit stuffing is a technique implemented in data communication to restrict specific bit patterns in the data to stop from being confused with control signals, such as frame delimiters. This technique is particularly helpful in protocols that send data as a continuous stream of bits, where sure bit patterns are occupied for signaling the start or end of a frame.

Code:

```
#include <stdio.h>
#include <string.h>

int main() {
    char data[100], stuffed[200];
    char flag = 'F';
    char escape = 'E';
    int i, j = 0;

    printf("Enter the data: ");
    scanf("%s", data);

    for (i = 0; data[i] != '\0'; i++) {
        // If data contains FLAG or ESC character,
        // insert an ESC character before it.
        if (data[i] == flag || data[i] == escape) {
            stuffed[j++] = escape;
        }

        stuffed[j++] = data[i];
    }

    stuffed[j] = '\0';

    printf("\nOriginal Data : %s\n", data);
    printf("Stuffed Data : %c%s%c\n", flag, stuffed, flag);

    return 0;
}
```

Output:

Enter the data: abcde

Original Data : abcde

Stuffed Data : FabcdeF

Practical-8

AIM: Write a C program to demonstrate the concept of PARITY CHECKING.

Code:

```
#include <stdio.h>

int main()
{
    int data[20], n, i, count = 0, parity;

    printf("Enter the number of data bits: ");
    scanf("%d", &n);

    printf("Enter the data bits (0 or 1):\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &data[i]);

        if (data[i] == 1)
            count++;
    }

    // Generate even parity bit
    parity = count % 2;

    printf("\nParity bit = %d", parity);

    printf("\nTransmitted data: ");
    for (i = 0; i < n; i++)
        printf("%d", data[i]);

    printf("%d", parity);

    // Check parity
    count += parity;

    if (count % 2 == 0)
        printf("\n\nParity Check: No error detected.");
}
```

```
else
    printf("\n\nParity Check: Error detected.");

return 0;
}
```

Output:

Enter the number of data bits: 7

Enter the data bits (0 or 1):

1 0 1 1 0 0 1

Parity bit = 0

Transmitted data: 10110010

Parity Check: No error detected.

Practical-9

AIM: Write a program to demonstrate the concept of Error Detection Method CRC.

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_LEN 100

// Function to perform Modulo-2 Division (XOR operation)
void modulo2Division(char *dividend, const char *divisor, char *remainder) {
    int divisor_len = strlen(divisor);
    int dividend_len = strlen(dividend);

    // Copy dividend into a temporary workspace
    char temp[MAX_LEN];
    strcpy(temp, dividend);

    // Perform bitwise XOR division loop
    for (int i = 0; i <= dividend_len - divisor_len; i++) {
        // If the leading bit is '1', XOR with the divisor
        if (temp[i] == '1') {
            for (int j = 0; j < divisor_len; j++) {
                temp[i + j] = (temp[i + j] == divisor[j]) ? '0' : '1';
            }
        }
        // If the leading bit is '0', we effectively XOR with 0 (do nothing, just move forward)
    }

    // The last (divisor_len - 1) bits form the remainder
    strncpy(remainder, temp + (dividend_len - (divisor_len - 1)), divisor_len - 1);
    remainder[divisor_len - 1] = '\0'; // Null-terminate string
}

int main() {
    char data[MAX_LEN], divisor[MAX_LEN];
    char appended_data[MAX_LEN], remainder[MAX_LEN];
    char transmitted_msg[MAX_LEN], received_msg[MAX_LEN];
```

```
printf("--- SENDER SIDE ---\n");
printf("Enter binary data string: ");
scanf("%s", data);
printf("Enter generator polynomial (divisor): ");
scanf("%s", divisor);

int data_len = strlen(data);
int divisor_len = strlen(divisor);

// Step 1: Initialize appended data with the original data
strcpy(appended_data, data);

// Step 2: Append (divisor_len - 1) zeros to the data
for (int i = 0; i < divisor_len - 1; i++) {
    strcat(appended_data, "0");
}

// Step 3: Perform Modulo-2 division to calculate CRC checksum
modulo2Division(appended_data, divisor, remainder);
printf("Calculated CRC Checksum (Remainder): %s\n", remainder);

// Step 4: Construct transmitted message (Original Data + CRC Checksum)
strcpy(transmitted_msg, data);
strcat(transmitted_msg, remainder);
printf("Transmitted Message (Data + CRC): %s\n\n", transmitted_msg);

printf("--- RECEIVER SIDE ---\n");
printf("Enter the received binary message: ");
scanf("%s", received_msg);

// Step 5: Perform Modulo-2 division on received message using the same divisor
char receiver_remainder[MAX_LEN];
modulo2Division(received_msg, divisor, receiver_remainder);

printf("Receiver Remainder: %s\n", receiver_remainder);

// Step 6: Verify if the remainder consists entirely of zeros
int error_detected = 0;
for (int i = 0; i < divisor_len - 1; i++) {
    if (receiver_remainder[i] == '1') {
        error_detected = 1;
        break;
    }
}
```

```
    }  
}  
  
if (error_detected) {  
    printf("Result: Error detected in data transmission! (Data is corrupted)\n");  
} else {  
    printf("Result: No error detected. (Data transmission successful!)\n");  
}  
  
return 0;  
}
```

Output:

--- SENDER SIDE ---

Enter binary data string: 100100

Enter generator polynomial (divisor): 1101

Calculated CRC Checksum (Remainder): 001

Transmitted Message (Data + CRC): 100100001

--- RECEIVER SIDE ---

Enter the received binary message: 1000100001

Receiver Remainder: 110

Result: ✖ Error detected in data transmission! (Data is corrupted)